

Laboratorio 2

Esquemas de detección y corrección de errores

Nombre	Carné
Daniel Chet	231177
Cristian Túnchez	231359

1. Descripción de la práctica

Se construyó una aplicación de mensajería que representa la comunicación entre un cajero automático y un servidor bancario sobre un canal no confiable. El cajero se implementó en Python y el servidor en Go. Ambos extremos pueden actuar como emisor y receptor sobre una misma conexión TCP y conservan la separación de cinco responsabilidades solicitada: Aplicación, Presentación, Enlace, Ruido y Transmisión.

Arquitectura por capas

- **Aplicación:** Solicita el texto, el algoritmo y la tasa de error; además, muestra el mensaje recibido o el error informado.
- **Presentación:** Convierte cada carácter a ASCII binario de ocho bits y realiza la decodificación inversa cuando la integridad lo permite.
- **Enlace:** Agrega, verifica y, en el caso de Hamming, corrige la redundancia asociada con el mensaje.
- **Ruido:** Voltea de forma independiente cada bit de datos y redundancia con la probabilidad indicada en cada envío.
- **Transmisión:** Empaqueta la trama con un encabezado de longitud explícita y la transporta sobre un socket TCP.

Protocolo de trama

El encabezado CC67 conserva versión, algoritmo y longitudes en orden de red. El cuerpo contiene solamente la trama de Enlace empaquetada MSB primero. El ruido no modifica el encabezado porque este pertenece a Transmisión y es necesario para reconstruir límites sobre el flujo TCP. En cambio, todos los bits de datos y redundancia están expuestos al canal.

Campo	Octetos	Función
Magic	4	ASCII CC67
Version / algoritmo / flags	4	1=Hamming, 2=CRC-32
Longitud original	4	Cantidad de bits sin redundancia
Longitud de trama	4	Cantidad de bits con redundancia
Cuerpo	variable	$\text{ceil}(\frac{\text{frame}_{\text{bits}}}{8}) \text{ octetos}$

Algoritmos

Hamming SEC coloca paridad par en posiciones potencia de dos y selecciona el menor r que cumple $m + r + 1 \leq 2^r$. El síndrome localiza un único bit alterado, coherente con el código sistemático de corrección de un error presentado por Hamming.

Caso manual:

datos 1011 -> código 0110011; error en posición 5 -> síndrome 5 -> corrección al voltear esa posición.

CRC-32 usa la forma reflejada 0xEDB88320 del polinomio 0x04C11DB7, init y xorout 0xFFFFFFFF. Los 32 bits calculados se concatenan al mensaje. El receptor recalcula y compara, una práctica consistente con el FCS de 32 bits documentado para enlaces y con el pre/post-acondicionamiento descrito por zlib. El vector ASCII 123456789 produce CBF43926. Para entradas de hasta 32 bits se agrega padding cero a la derecha solo durante el cálculo.

Vector conocido: ASCII "123456789" -> CRC-32 0xCBF43926.

2. Resultados

Se ejecutaron 15,000 transmisiones simuladas con semilla 3067: 300 repeticiones para cada una de 50 combinaciones. Se variaron dos algoritmos, mensajes de 1, 4, 16, 64 y 256 bytes y probabilidades por bit de 0%, 0.1%, 0.5%, 1% y 2%. Cada resultado se clasificó como recuperación exacta, rechazo informado o corrupción silenciosa.

Overhead

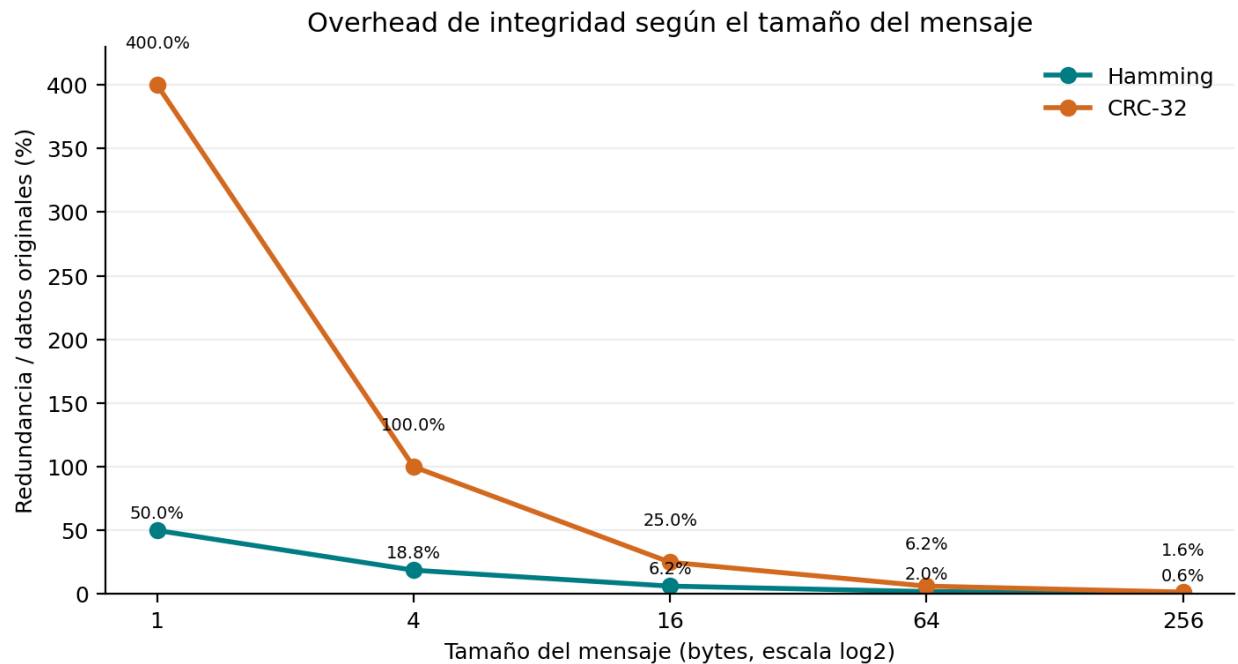


Figura 1. Redundancia relativa.

Mensaje	Hamming	CRC-32
8 Bits	4 Bits (50%)	32 Bits (400%)
32 Bits	6 Bits (18.8%)	32 Bits (100%)
128 Bits	8 Bits (6.2%)	32 Bits (25%)
512 Bits	10 Bits (2%)	32 Bits (6.2%)
2048 Bits	12 Bits (0.6%)	32 Bits (1.6%)

En un byte CRC-32 representa 400% de overhead, mientras Hamming usa cuatro bits (50%). A 256 bytes ambos costos son pequeños, pero Hamming permanece menor: 12 bits contra 32 bits

Recuperación Exacta

Recuperación exacta: algoritmo, longitud y ruido

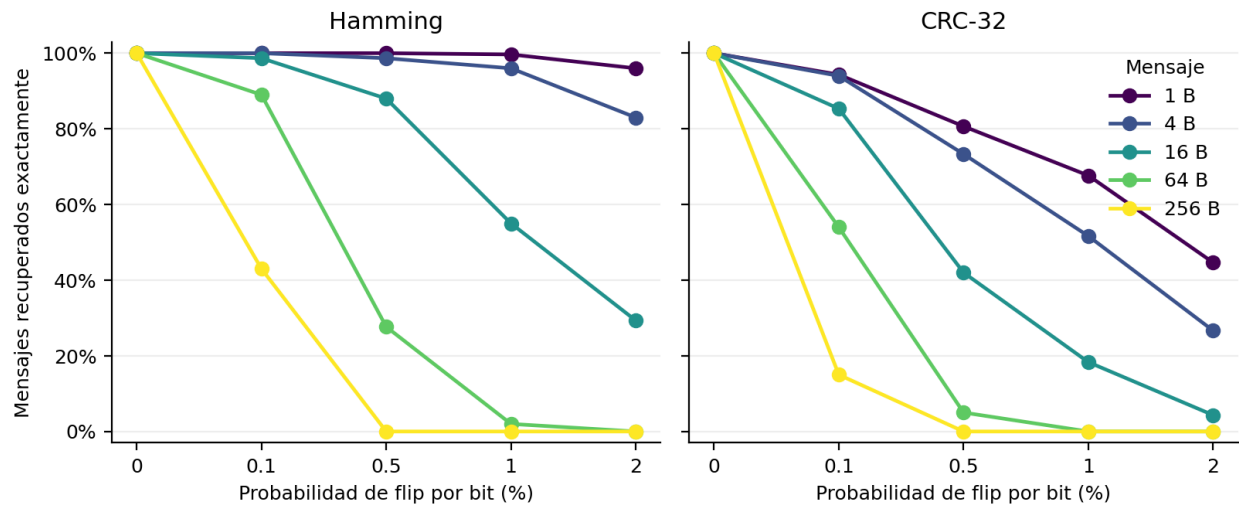


Figura 2. Proporción entregada exactamente.

La probabilidad de que una trama acumule varios flips aumenta tanto con la tasa de error como con su longitud. Por esa razón todas las curvas descienden con mensajes mayores. Hamming conserva una ventaja de entrega exacta cuando hay un único error, pero esa ventaja no implica que todo mensaje aceptado sea correcto.

Resultado visible para una longitud fija

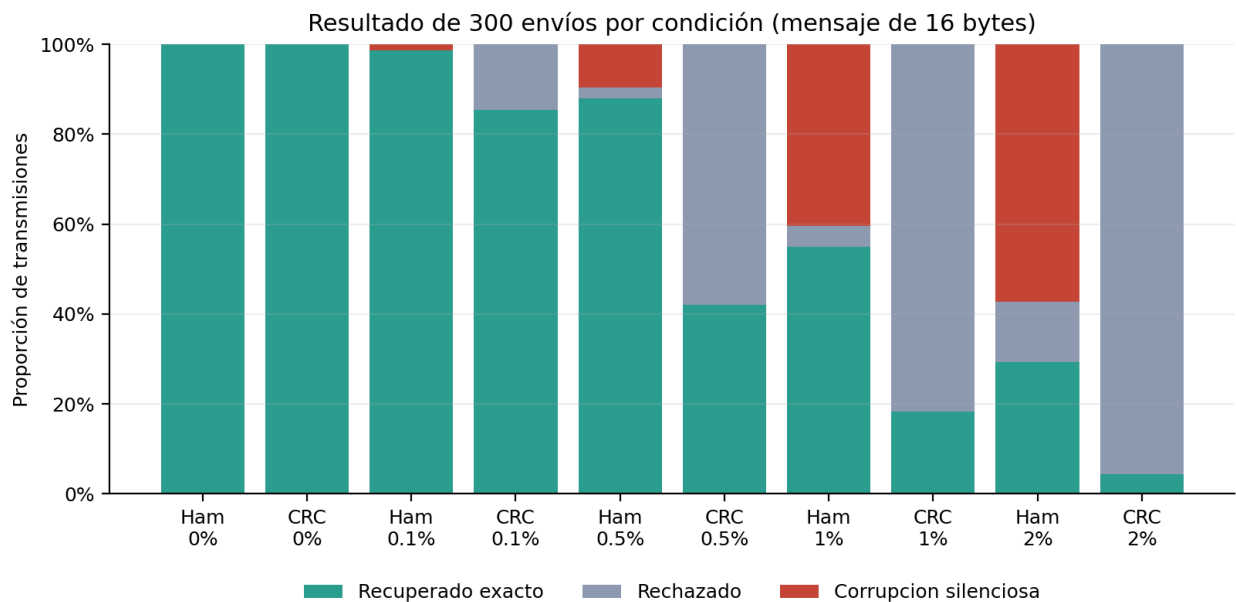


Figura 3. Distribución de resultados para mensajes de 16 bytes.

Algoritmo	p(bit)	Exacto	Rechazado	Silencioso	Flips prom.
Hamming	0.1%	98.7%	0%	1.3%	0.15
CRC-32	0.1%	85.3%	14.7%	0%	0.16
Hamming	0.5%	88%	2.3%	9.7%	0.61
CRC-32	0.5%	42%	58%	0%	0.80
Hamming	1%	55%	4.7%	40.3%	1.47
CRC-32	1%	18.3%	81.7%	0%	1.67
Hamming	2%	29.3%	13.3%	57.3%	2.60
CRC-32	2%	4.3%	95.7%	0%	3.14

Tabla 1. Resultados numéricos para 16 bytes.

3. Discusión

¿Qué algoritmo tuvo mejor funcionamiento?

Con 16 bytes y $p=0.1\%$, Hamming recuperó exactamente 98.7%, frente a 85.3% de CRC-32, gracias a su corrección de un flip. Sin embargo, CRC-32 convirtió la corrupción en un error visible: en estas pruebas su tasa silenciosa fue 0.0%. Para una aplicación bancaria, detectar y no mostrar un mensaje dañado suele ser preferible a aceptar un valor incorrecto, por lo que CRC-32 ofrece el comportamiento más seguro cuando puede solicitarse un reenvío.

¿Cuál acepta mayores tasas de error?

Hamming es más útil mientras la probabilidad de cero o un flip por palabra siga siendo importante; el costo es pequeño y la corrección es inmediata. Al subir a $p=2\%$ para 16 bytes, su recuperación exacta bajó a 29.3% y la corrupción silenciosa llegó a 57.3%. CRC-32 recuperó exactamente 4.3%, pero rechazó 95.7% en lugar de entregar contenido alterado. Por tanto, Hamming es flexible ante errores aislados; CRC-32 es más confiable para reconocer que un canal con muchos errores ya no permite entregar la trama.

¿Detección o corrección?

Conviene detección cuando existe un canal de retorno y el costo de retransmitir es menor que el riesgo de corregir mal, especialmente en transacciones. Conviene corrección cuando la latencia de ida y vuelta es alta, no hay retorno o se espera como máximo un error por palabra. En un sistema real puede combinarse detección fuerte, retransmisión y códigos por bloques.

Efecto de longitud y overhead

El CRC tiene overhead absoluto constante, por lo que su porcentaje cae rápidamente al crecer el mensaje. Hamming agrega r de forma logarítmica y fue más eficiente en todas las longitudes ensayadas. No obstante, usar una sola palabra Hamming para todo el mensaje mantiene capacidad de corregir solo un bit total: aumentar la longitud eleva la probabilidad de errores múltiples. Dividir en bloques mejoraría esa propiedad, pero cambiaría el esquema pedido y no se implementó.

Amenazas a la validez

El canal simulado supone flips independientes con probabilidad constante; no modela ráfagas, pérdidas ni latencia. Las 300 repeticiones por condición producen variación muestral y los resultados exactos dependen de la semilla declarada. La comparación mide integridad de una trama, no rendimiento temporal ni reintentos. La corrupción del encabezado de Transmisión queda fuera del modelo porque el requisito aplica ruido a la salida de Enlace

4. Conclusiones

1. La arquitectura en capas permitió cambiar algoritmo, ruido y transporte sin mezclar la conversión ASCII ni la interfaz de usuario. El protocolo binario de longitud explícita resolvió la falta de límites de mensaje en TCP y mantuvo interoperabilidad entre Python y Go.
2. El Hamming genérico cumplió la desigualdad de paridad y corrigió cualquier error simple probado con overhead bajo. Su límite fundamental es que dos o más flips pueden convertirse en una corrección equivocada, por lo que no debe interpretarse el síndrome como garantía de integridad bajo ruido alto.
3. CRC-32 agregó 32 bits independientemente del tamaño y detectó las tramas alteradas de la muestra sin intentar repararlas. Su overhead es alto para mensajes muy cortos, pero disminuye proporcionalmente y su rechazo explícito resulta apropiado para información bancaria.

5. Referencias

- R. W. Hamming. "Error Detecting and Error Correcting Codes." The Bell System Technical Journal, vol. 29, no. 2, pp. 147-160, 1950. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>

- W. Simpson, ed. PPP in HDLC-like Framing, RFC 1662, Internet Engineering Task Force, julio de 1994. <https://www.rfc-editor.org/rfc/rfc1662.html>
- J.-I. Gailly y M. Adler. zlib 1.3.1 Manual: Checksum Functions, 2024. <https://www.zlib.net/manual.html>